

Programación con *sockets* - Introducción

La creación de un programa en C++ que usa *sockets* para comunicarse a través de una infraestructura de red, con el objetivo de enviar o recibir información, implica trabajar con múltiples estructuras de datos e invocar a diferentes servicios ofrecidos por el sistema operativo.

La mayoría de los servicios para trabajar con *sockets* son idénticos para ambas versiones del protocolo IP, IPv4 e IPv6, y suelen precisarse los mismos pasos: crear el *socket*, usarlo para abrir una conexión, enviar o recibir datos, etc. Esta es la razón de que existan estructuras de datos tanto específicas para cada versión como otras comunes. Las funciones, por el contrario, son en su mayor parte comunes.

Nota

El objetivo de esta primera práctica será familiarizarnos con los aspectos genéricos de la programación con *sockets*. Los específicos, según el tipo de dirección a emplear, IPv4 o IPv6, o bien el tipo de *socket*, UDP o TCP, los abordaremos en prácticas posteriores.

POSIX *sockets*

El primer paso, antes de iniciar estas prácticas, será establecer qué servicios de *sockets* emplearemos. Cada sistema operativo ofrece los suyos, a los que hay que sumar los de más alto nivel que ofrecen plataformas como la de Java, Microsoft .NET y otros lenguajes más modernos. Existen, por tanto, **múltiples conjuntos de servicios** para programación con *sockets*, según el lenguaje de programación que elijamos, la plataforma sobre la que se ejecuten los programas, el sistema operativo, etc.

De toda esa oferta, vamos a elegir los conocidos como *POSIX sockets*. POSIX (*Portable Operating System Interface for uni-X*) define un conjunto de estándares, controlados por la IEEE Computer Society, con el objetivo de que ciertas interfaces de programación o API sean **compatibles entre sistemas y plataformas**. La ventaja de usar esos servicios estriba en que el código que escribamos (en lenguaje C o C++) para trabajar con *sockets* podrá ser compilado, con mínimas o nulas modificaciones, para su ejecución en los sistemas operativos GNU/Linux, Windows y macOS.

Nota

Los *POSIX sockets* son el estándar actual a la hora de trabajar con *sockets* desde lenguajes como C y C++.

Herramientas y compilación

Las herramientas fundamentales que precisaremos para completar esta y las demás prácticas relacionadas con programación con *sockets* serán:

- **Un editor de código.** Puedes elegir el que más cómodo te resulte, desde **nano** o **vim** hasta **VSCode** o el IDE de **XCode** en macOS. Lo único importante es que permita trabajar con el código de los programas.
- **Un compilador de C++.** El código de los programas será procesado, y generará un ejecutable, con el compilador GCC para C++ (**g++**), disponible tanto en GNU/Linux como macOS. En Windows las alternativas son varias: Visual C++ (será preciso introducir cambios puntuales en el

código), WSL (*Windows Subsystem for Linux*, que ofrece un entorno como el de Linux y permite usar `g++`) o una solución como `MingW` que facilita un entorno para usar GCC y generar ejecutables Windows.

Aviso

Si usas una máquina virtual, asegúrate de configurarla para que tengas acceso a Internet a fin de poder probar los distintos programas propuestos en los cuadernos de prácticas.

Los archivos donde se almacena el código tendrán extensión `.cpp`, lo cual les identifica como código fuente C++. Para su compilación, desde la *shell*, usaremos el comando `g++ archivo.cpp -o archivo -std=c++2a`. Esto producirá un ejecutable llamado `archivo` que podremos ejecutar introduciendo `./archivo` en la consola. La opción `-std=c++2a` indica al compilador la versión del estándar C++ que queremos usar.

Nota

En estos cuadernos de prácticas se asume que **trabajaremos desde la línea de comandos o *shell*** de GNU/Linux. Esta es nativa si tenemos instalado ese sistema operativo. La funcionalidad es equivalente desde la terminal de macOS. En el caso de Windows el método recomendado es instalar WSL.

Comprobación de compilación y ejecución

Lo primero que debes hacer es asegurarte de que tienes las herramientas apropiadas y, por tanto, **no hay problema para compilar un programa** en C++ y ejecutarlo. Para ello, reproduce los siguientes pasos:

1. Abre tu editor e introduce el código mostrado a continuación, pero **sin incluir los números que hay delante de las líneas**. Guarda el archivo con el nombre `base.cpp`:

```
1: #include <iostream>
2: #include <vector>

3: using namespace std;

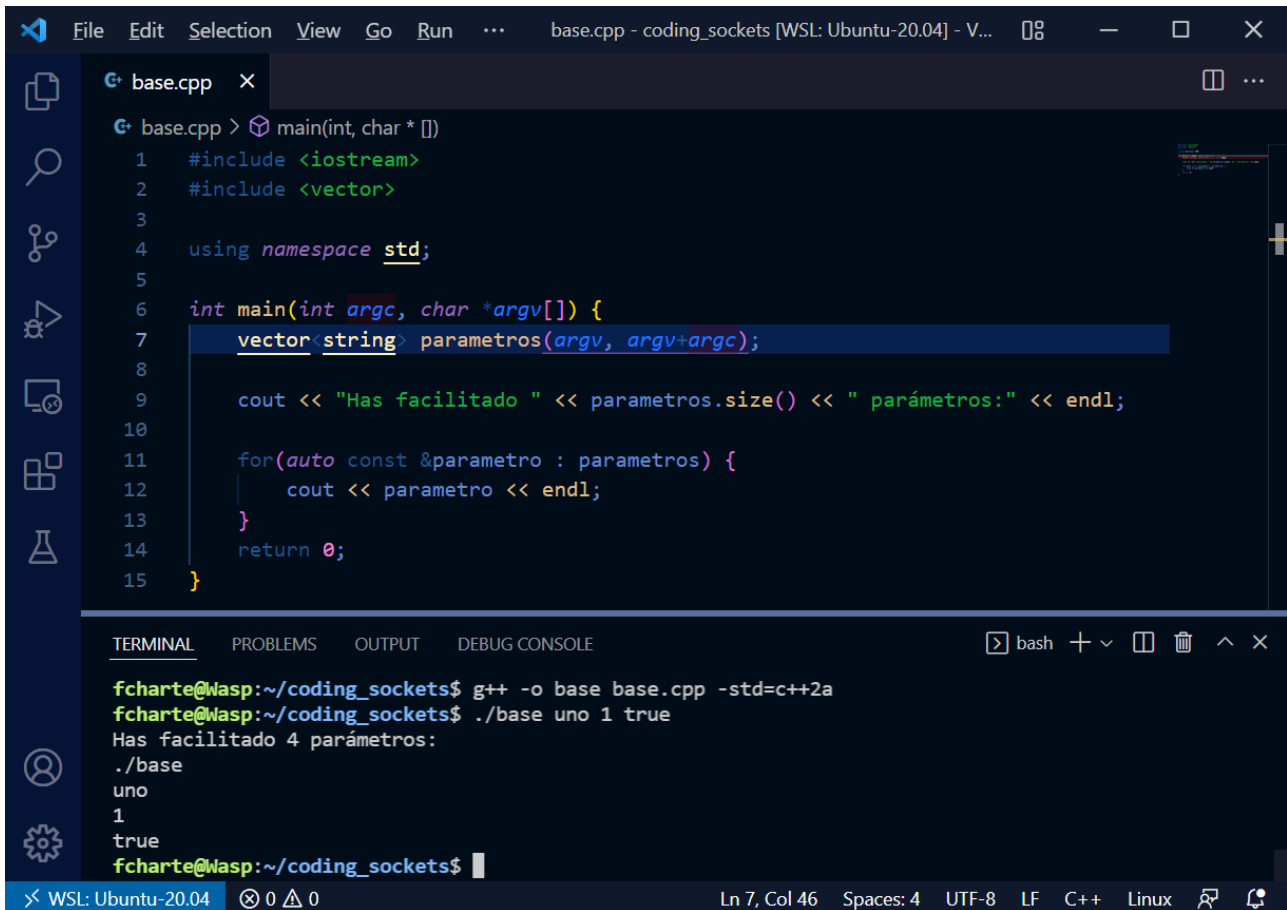
4: int main(int argc, char *argv[]) {
5:     vector<string> parametros(argv, argv+argc);

6:     cout << "Has facilitado " << parametros.size()
           << " parámetros:" << endl;

7:     for(auto const &parametro : parametros) {
8:         cout << parametro << endl;
9:     }

9:     return 0;
}
```

2. Desde la *shell*, encontrándote en el mismo directorio donde has guardado el archivo anterior, **compila el programa** con la siguiente orden: `g++ base.cpp -o base -std=c++2a`.
3. En la misma terminal introduce `./base uno 2 true`. Deberías ver aparecer en la consola una salida como la de la siguiente imagen, correspondiente a VSCode con el código en el editor y la consola debajo.



```
base.cpp > main(int, char * [])
1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5
6  int main(int argc, char *argv[]) {
7      vector<string> parametros(argv, argv+argc);
8
9      cout << "Has facilitado " << parametros.size() << " parámetros:" << endl;
10
11     for(auto const &parametro : parametros) {
12         cout << parametro << endl;
13     }
14     return 0;
15 }
```

```
fcharte@wasp:~/coding_sockets$ g++ -o base base.cpp -std=c++2a
fcharte@wasp:~/coding_sockets$ ./base uno 1 true
Has facilitado 4 parámetros:
./base
uno
1
true
fcharte@wasp:~/coding_sockets$
```

Figura 1. Programa en VSCode. Compilación y ejecución en la consola.

Este programa muestra cómo convertir los parámetros facilitados como entrada al programa en un **vector de cadenas** al estilo de C++ (tipo `vector<string>`), de forma que después es fácil acceder a los mismos. Atendiendo a la numeración de la líneas de código, el objetivo de cada sentencia es el siguiente:

1. Incluye el archivo de cabecera `iostream` que nos permite utilizar los objetos `cout` y `cin`, así como los operadores `<<` y `>>`.
2. Incluye el archivo de cabecera `vector` en el que está definida la plantilla `vector<tipo>`.
3. Incorpora al ámbito de nuestro programa los elementos definidos en el espacio de nombres `std`, de forma que podamos usarlos sin necesidad de escribir el prefijo: `cout` en lugar de `std::cout`, `vector` en lugar de `std::vector`, etc.
4. Firma estándar de la función `main()` en un programa C++, con el tipo `int` como valor de salida y parámetros de entrada `argc` (número de argumentos facilitados al invocar al programa) y `argv`

vector de punteros `char` con el texto de los argumentos.

5. Toma los argumentos de entrada y los almacena en un vector de cadenas llamado `parámetros`, lo cual hace más fácil trabajar con ellos.
6. Muestra en la consola el número de parámetros entregados al invocar al programa, dato obtenido con el método `size()` del vector donde están almacenados.
7. Bucle `for` en el que se recorre el vector `parametros`, asignando uno de sus elementos a la variable `parametro` en cada iteración. Con `auto` se infiere el tipo de `parametro`. También podríamos escribir `string const ¶metro`.
8. Muestra el parámetro por la consola.
9. Sale de la función `main()`, terminando la ejecución del programa y devolviendo el código `0`. Este es el habitual cuando no hay errores.

Nota

Dado que ya has cursado varias asignaturas en las que se usa el lenguaje C++, en principio deberías comprender los fragmentos de código que se irán proponiendo en estas prácticas. Ante cualquier duda consulta con tu profesor/a.

EJERCICIO 1

Modifica el programa anterior de forma que no se utilice el tipo `vector` y, en su lugar, usa directamente los parámetros `argc` y `argv` recibidos por la función `main()`. El resultado generado por el programa debería seguir siendo el mismo. **Detalla a continuación** qué líneas del programa has cambiado (código original) y de qué forma (nuevo código).

El concepto de *socket*

Para que un programa tenga la capacidad de comunicarse con otros, a través de una infraestructura de red mediante cualquiera de los **protocolos de la pila TCP/IP**, ya sea de la capa de red, de enlace o de aplicación, precisaremos siempre un *socket*.

Un *socket* es, en cierta manera, como el identificador (*handle*) que se obtiene al abrir un archivo. Dicho identificador representa una conexión con el archivo y tiene una cierta configuración: lectura, escritura o ambos, creación del archivo, etc. De forma análoga, al crear un *socket* también **estableceremos su configuración**. Según esta, tendremos un *socket* TCP, uno UDP o un *raw socket* o *socket* IP.

¿Cómo crear un *socket*?

El primer paso para enviar o recibir datos a través de una interfaz de red será la creación de un *socket* adecuado a las necesidades que tengamos: familia de protocolos que se quiere usar, tipo de *socket*, etc.

Una vez que se cuenta con el *socket*, representado por un identificador de tipo `int`, este puede emplearse para quedar a la escucha, establecer una conexión, enviar datos y recibirlos, etc. Esas son operaciones que conoceremos más adelante.

La función `socket()`

La función `socket()` es la encargada de crear un nuevo *socket*, para lo cual deberá asignar los recursos necesarios y llevar a cabo un proceso de inicialización. Esta función está definida en el archivo de cabecera `sys/socket.h`. Al invocarla deberemos facilitar tres argumentos:

- **Dominio:** un `int` que establece la familia de protocolos que se usará en la comunicación. Todos ellos están representados por constantes del tipo `AF_XXX` y la lista es considerablemente amplia.
- **Tipo:** otro `int` que selecciona un tipo de *socket* de entre los permitidos en la familia indicada por el parámetro anterior. Por ejemplo, para la familia `AF_INET`, que corresponde a IPv4, son tipos válidos `SOCK_STREAM` (un *socket* de transporte orientado a conexión: TCP), `SOCK_DGRAM` (un *socket* de transporte no orientado a conexión: UDP) y `SOCK_RAW` (un *socket* al nivel de la capa de red), entre otros.
- **Protocolo:** un tercer `int`, en este caso para elegir uno de los protocolos disponibles para el tipo de *socket* indicado. Tipos comunes son `IPPROTO_TCP` e `IPPROTO_UDP`. También puede tomar el valor `0`, en caso de que para el tipo de *socket* indicando como segundo argumento solo exista un protocolo posible.

Nota

Recuerda que siempre puedes usar el comando `man función` para acceder a la documentación electrónica de cualquier función y conocer todos los detalles.

El valor devuelto por la función será un entero que identifica al *socket*, de manera análoga a los identificadores de archivo o *handle*. Si ese valor es `-1` indicará que **no ha sido posible la creación**, en cuyo caso la variable `errno` contendrá el código de error.

Cierre del *socket*

Cuando el *socket* haya dejado de sernos útil deberíamos liberar los recursos que ocupa, aquellos reservados por la función `socket()`. Con este fin se facilita el identificador del *socket* a la función `close()`, como haríamos para cerrar un archivo. Al operar con *sockets* en realidad no cerramos nada, porque la función `socket()` no abre una conexión, de forma que `close()` se limita a liberar recursos.

Plantilla de código para crear y liberar un *socket*

El mostrado a continuación es el código mínimo para crear un *socket* con una configuración concreta, verificar que ese proceso ha tenido éxito y, por último, liberarlo cuando haya dejado de ser útil.

```
#include <stdexcept>
#include <cstring>
#include <iostream>
#include <sys/types.h>
#include <sys/socket.h>
#include <unistd.h>
#include <netinet/in.h>
#include <netinet/ip.h>

using namespace std;
```

```
int main(int argc, char *argv[]) {
    int misocket;

    if(misocket = socket(AF_INET, SOCK_RAW, IPPROTO_RAW); misocket == -1) {
        throw runtime_error(strerror(errno));
    }
    cout << "Se ha creado el socket satisfactoriamente" << endl;
    close(misocket);
    return 0;
}
```

```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE

fcharte@wasp:~/coding_sockets/02_socketsip$ g++ -o crea_socket crea_socket.cpp -std=c++2a
fcharte@wasp:~/coding_sockets/02_socketsip$ ./crea_socket
terminate called after throwing an instance of 'std::runtime_error'
  what(): Operation not permitted
Aborted
fcharte@wasp:~/coding_sockets/02_socketsip$ sudo ./crea_socket
[sudo] password for fcharte:
Se ha creado el socket satisfactoriamente
fcharte@wasp:~/coding_sockets/02_socketsip$
```

Figura 2. La ejecución de este programa precisa de permisos de superusuario.

En la Figura 2 se aprecia que, tras compilar el programa, al ejecutarlo sin más, con nuestras credenciales de usuario estándar, obtenemos un error: `Operation not permitted`. El segundo intento, en el que se ha recurrido al comando `sudo` para ejecutar el programa con credenciales de superusuario, sí ha funcionado sin problemas.

Configuración de ejecución de un programa que usa *sockets*

Ejecutar un programa con privilegios de superusuario **no es recomendable**. Si dicho programa tuviese algún fallo, o pudiese ser atacado de alguna manera, dichos privilegios le permitirían realizar cualquier operación sobre el sistema, pudiendo comprometer su estado y toda la información que almacena.

Hay determinadas funcionalidades, no obstante, que no se permiten a los programas ejecutados con privilegios estándar. Entre ellos están la administración de las interfaces de red o el **uso de sockets con configuraciones no habituales**. Se considera habitual el uso, por parte de un programa cualquiera, de *sockets* TCP y UDP con puertos por encima del 1024. Los primeros 1024 puertos, así como el uso de *sockets* con otros protocolos de capas inferiores, están en principio vedados.

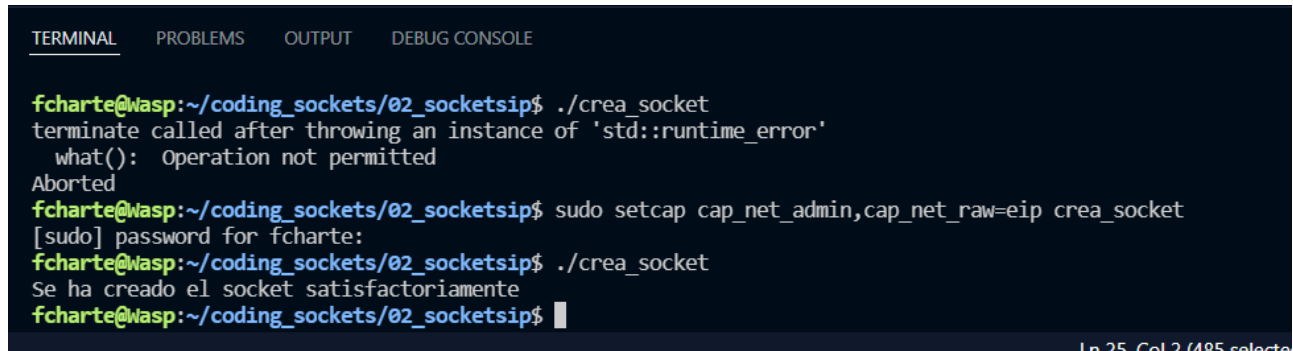
A pesar de lo dicho, hay programas que necesitan operar con protocolos de más bajo nivel y/o usar puertos por debajo del 1024. ¿Cómo conseguir que tengan acceso a esa funcionalidad sin otorgarles privilegios de superusuario? La respuesta es **asignar a ese programa los permisos específicos** que necesita (algo que debe hacer el superusuario), de forma que al ejecutarse pueda realizar dichas operaciones sin, por ello, tener privilegios absolutos como ocurre al ser superusuario.

El comando `setcap` es el encargado de gestionar esos permisos. En nuestro caso se precisan dos: `cap_net_admin` y `cap_net_raw`, lo cual nos permitirá administrar las interfaces de red y crear *sockets*

de tipo *raw* (bajo nivel, capa de red y enlace). La orden a ejecutar para otorgar dichos permisos a un programa sería la siguiente:

```
sudo setcap cap_net_admin,cap_net_raw=pei ejecutable
```

En la siguiente imagen puede apreciarse el error que se obtiene al ejecutar el programa antes de usar **setcap** y el resultado satisfactorio posterior.



```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE

fcharte@wasp:~/coding_sockets/02_socketsip$ ./crea_socket
terminate called after throwing an instance of 'std::runtime_error'
  what(): Operation not permitted
Aborted
fcharte@wasp:~/coding_sockets/02_socketsip$ sudo setcap cap_net_admin,cap_net_raw=eip crea_socket
[sudo] password for fcharte:
fcharte@wasp:~/coding_sockets/02_socketsip$ ./crea_socket
Se ha creado el socket satisfactoriamente
fcharte@wasp:~/coding_sockets/02_socketsip$
```

Figura 3. Establecemos la configuración de ejecución del programa para evitar hacerlo como superusuario.

Nota

Cada vez que compilamos un programa, el compilador genera un nuevo archivo ejecutable y elimina el anterior. Esto implica que cualquier configuración de permisos previa se pierde. Es por ello que deberemos usar la orden **setcap**, para fijar la configuración del ejecutable, después de cada compilación.

EJERCICIO 2

Usa el comando `man socket` para acceder a la documentación de dicha función. Al inicio de la misma encontrarás los detalles relativos a las constantes que pueden emplearse para los tres argumentos que precisa. También puedes hacer `cat /etc/protocols` para obtener una lista de los números de protocolo considerados por esta función. Ahora modifica el programa para crear un *socket* TCP y, a continuación, responde a las siguientes cuestiones:

1. ¿Qué valores has usado como parámetros a la hora de invocar a la función **socket()**? Explica el significado de cada uno de ellos.
2. ¿Es posible ejecutar el programa sin permisos especiales? ¿Por qué?
3. ¿Podrías indicar el protocolo (último parámetro) de otra forma? ¿Cómo?

Orden de los bytes en las palabras

Ciertos campos de las cabeceras IPv4 e IPv6, que ya hemos estudiado en teoría, se almacenan como valores enteros de 16 bits (el número de puerto o la suma de comprobación) o de 32 bits (direcciones IPv4). Estos campos son palabras o dobles palabras, compuestas de una **secuencia de dos o cuatro bytes** respectivamente.

Es importante tener en cuenta **el orden en que se almacenan esas secuencias** en la memoria. Ello dependerá de la **arquitectura del microprocesador** de nuestro ordenador, que puede ser *big endian* (la mayoría de procesadores RISC, en nuestros móviles, tabletas y servidores, lo son) o *little endian* (todos los procesadores x86 y x64, los más habituales en ordenadores portátiles y de sobremesa). Hemos de tener presente que **en Internet los datos numéricos se transfieren en formato *big endian*** por lo que, si nuestro ordenador usa un orden distinto, es preciso llevar a cabo las conversiones apropiadas, de lo contrario los datos enviados o recibidos se interpretarían de manera incorrecta.

Nota

Que la arquitectura de un procesador sea *big endian* o *little endian* determina el orden en el que se almacenan en memoria (y por tanto en flujos de datos) los bytes de una palabra de 16 bits, las palabras de una doble palabra (32 bits), etc. En la arquitectura *little endian* el byte de menor peso (LSB) se dispone en la memoria antes del de mayor peso (MSB), mientras que en *big endian* el orden es el *natural*, inverso al anterior.

A fin de que podamos escribir código que **funcione siempre de forma correcta** al operar con números de más 8 bits, en el archivo de cabecera `in.h` se definen cuatro funciones llamadas `htons()`, `htonl()`, `ntohs()` y `ntohl()`. La inicial `h` (*host*) indica que el valor que se va a facilitar está en el formato de nuestro equipo y se quiere transformar al de la red, mientras que `n` (*network*) efectúa la conversión inversa, al convertir un valor recibido de la red al formato del ordenador que ejecuta el programa. El sufijo `s` (*short*) o `l` (*long*) determina si el valor a convertir es un entero de 16 o de 32 bits.

Las reglas generales de uso de estas funciones son las siguientes:

- Al recibir un paquete desde la red, incluso si es desde nuestra propia LAN, los campos que son números enteros de 16 y 32 bits vendrán en formato *big endian*. Por ello, recurriremos a las funciones `ntohs()` y `ntohl()` para convertirlos antes de trabajar con ellos.
- Al enviar un paquete desde nuestro ordenador, con independencia de cuál sea su arquitectura, convertiremos los números enteros de 16 y 32 bits al formato de la red, para lo cual emplearemos las funciones `htons()` y `htonl()`.

EJERCICIO 3

Escribe un programa que realice las siguientes tareas:

1. Solicitar por consola dos números enteros, uno de 16 bits y otro de 32 bits. Conviértelos a los respectivos tipos `short` e `int` si es necesario.
2. Usar las funciones de conversión `ntohs()` y `ntohl()` para convertirlos al formato de tu máquina y muestra el resultado en la consola. Indica cuál es el resultado obtenido y razona brevemente sobre por qué se ha obtenido.
3. Usar las funciones de conversión `htons()` y `htonl()` para convertirlos al formato de Internet y muestra el resultado en la consola. Indica cuál es el resultado obtenido y razona brevemente sobre por qué se ha obtenido.